

CHAPTER 4.1 - basic import

```
import pandas as pd
data = pd.Series ([1,2,3],index=['a','b','c'])
print (data['b'])
data['c']=4
```

2

```
data[['a','c','b']]
```

```
a    1
c    4
b    2
dtype: int64
```

```
data1 = {"x": 100, "y": 200}
count =["x","y"]
data2 = pd.Series(data1,index =count)
data2
```

```
x    100
y    200
dtype: int64
```

```
data2.name = "value"
data2.index.name = "count"
data2 =pd.DataFrame(data2)
data2
```

```
      value
count
x      100
```

```
import pandas as pd
data3 = {
    "state":{1:"a",2:"b",3: "c"},
    "name"  :{1: "x",2: "y",3: "z"}
}
data3 =pd.DataFrame(data3)
data3
```

```
   state name
1      a    x
2      b    y
3      c    z
```

```
data_need = {"state" : data3["state"][:1],"name" : data3["name"][:1]}
pd.DataFrame(data_need)
```

```
   state name
1      a    x
```

```

data3.index.name = "no."
data3.columns.name = "all"
data3.values

array([[ 'a', 'x'],
       [ 'b', 'y'],
       [ 'c', 'z']], dtype=object)

"elijah" in data3.columns

False

```

CHAPTER 4.2 sorting value+fill value

```

import numpy as np
import pandas as pd
data4 = {
    "state":{"m":"a","n":"b","o": "c"},
    "name" : {"m": "x","n": "y","o": "z"}
}
data4 =pd.DataFrame(data4)
data4

  state name
m      a   x
n      b   y
o      c   z

data4.iloc[0,0] =np.nan
data4

  state name
m    NaN   x
n      b   y
o      c   z

data4.dropna()

  state name
n      b   y
o      c   z

df = pd.DataFrame([[1.4, np.nan], [7.1, -4.5],[np.nan, np.nan], [0.75,
-1.3]]),
                    index=[ 'a', 'b', 'c', 'd'],columns=[ 'one', 'two'])
df

  one two
a  1.40 NaN
b  7.10 -4.5

```

```
c    NaN    NaN
d    0.75 -1.3
```

CHAPTER 5.1 - missing value+filter

```
import pandas as pd
import numpy as np
data = pd.DataFrame([[1., np.nan, 3.], [1., np.nan, np.nan], [np.nan,
np.nan, np.nan], [np.nan, np.nan, 3.]])
data
#data.dropna(axis =1,how ='all')
#data.dropna(axis =1,thresh =1)
#data.fillna({0:100,1:200,2:300},inplace=True)
#data.fillna(method ='ffill',limit = 2)
```

	0	1	2
0	1.0	NaN	3.0
1	1.0	NaN	NaN
2	NaN	NaN	NaN
3	NaN	NaN	3.0

CHAPTER 5.2 - remove duplicate value+filter

```
import pandas as pd
data = pd.DataFrame({'k1': ['a', 'b'] * 3 + ['b'], 'k2': [1, 1, 2, 3,
3, 4, 4]})
data
#data.drop_duplicates() // (["k2"]) // (['k1', 'k2'], keep='last')
```

	k1	k2
0	a	1
1	b	1
2	a	2
3	b	3
4	a	3
5	b	4
6	b	4

CHAPTER 5.2.2 - mapping

```
import pandas as pd
data = pd.DataFrame({'vege': ['cabbage', 'carrot',
'cabbage', 'Lettuce', 'potato', 'Cabbage',
'lettuce', 'tomato', 'cucumber'],
'weight': [4, 3, 12, 6, 7.5, 8, 3, 5, 6]})
vege_to_seller = {'cabbage': 'David', 'carrot': 'David', 'lettuce':
'Ahmad', 'potato': 'Ahmad',
'tomato': 'David', 'cucumber': 'Tina'}
lower = data['vege'].str.lower()
```

```
data['seller'] = lower.map(vege_to_seller)
data
```

	vege	weight	seller
0	cabbage	4.0	David
1	carrot	3.0	David
2	cabbage	12.0	David
3	Lettuce	6.0	Ahmad
4	potato	7.5	Ahmad
5	Cabbage	8.0	David
6	lettuce	3.0	Ahmad
7	tomato	5.0	David
8	cucumber	6.0	Tina

CHAPTER 5.2.3 - replace

```
import pandas as pd
data = pd.Series([1, 2, 3, 4,5,6])
data
#data.replace(1,100) // ([1,2],100)
```

```
0    1
1    2
2    3
3    4
4    5
5    6
dtype: int64
```

CHAPTER 5.2.4 - rename

```
import pandas as pd
data = pd.DataFrame(np.arange(12).reshape((3, 4)), index=['a', 'b', 'c'],
                    columns=['v', 'x', 'y', 'z'])
data.rename(index={'a': 'A'},columns={'v': 'V'})
```

	V	x	y	z
A	0	1	2	3
b	4	5	6	7
c	8	9	10	11

CHAPTER 5.2.5 - descretization+binning

```
import pandas as pd
ages = [20, 22, 25, 27, 21, 23, 37, 31, 61, 45, 41, 32]
bins = [18, 25, 35, 60, 100]
cats = pd.cut(ages, bins)
# cats
```

```
# cats.codes
cats.categories
# pd.value_counts(cats)

IntervalIndex([(18, 25], (25, 35], (35, 60], (60, 100]],
dtype='interval[int64, right]')
```

CHAPTER 5.3.1 - string manipulation

```
import pandas as pd
val = 'a,b, c'
# val.split(',')
# val.count(',')
# val.replace(',', ' ')
# pieces = [x.strip() for x in val.split(',')] #空格去掉
# first, second, third = pieces #分类

['a', 'b', ' c']
```

CHAPTER 5.3.2 - regular expression

```
import re
text = "a    b\t c  \td"
regex = re.compile('\s+')
regex.split(text)

<>:3: SyntaxWarning: invalid escape sequence '\s'
<>:3: SyntaxWarning: invalid escape sequence '\s'
C:\Users\User\AppData\Local\Temp\ipykernel_25720\18366913.py:3:
SyntaxWarning: invalid escape sequence '\s'
    regex = re.compile('\s+')

['a', 'b', 'c', 'd']
```

CHAPTER 5.3.3 - string function

```
import pandas as pd
data = {'Dave': 'dave@google.com', 'Steve': 'steve@gmail.com', 'Rob':
'rob@gmail.com', 'Wes': np.nan}
data = pd.Series(data)
pattern = r'[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,4}'
# data.str.contains('gmail')
# data.str.findall(pattern, flags=re.IGNORECASE)

data.str.findall(pattern, flags=re.IGNORECASE)

Dave      [dave@google.com]
Steve     [steve@gmail.com]
Rob       [rob@gmail.com]
```

```
Wes                                     NaN
dtype: object
```

CHAPTER 6.1 - data wrangling(hierarchical indexing)

```
import pandas as pd
import numpy as np
data = pd.Series(np.random.randn(9), index=[['a', 'a', 'a', 'b', 'b',
'c', 'c', 'd', 'd']],
                 [1, 2, 3, 1, 3, 1, 2, 2,
3]))
data
# data.loc[['b', 'd']]
# data.unstack()
```

a	1	1.334725
	2	1.103066
	3	-0.463100
b	1	0.387713
	3	-0.406431
c	1	-1.034836
	2	0.802655
d	2	-1.394240
	3	0.004548

dtype: float64

CHAPTER 6.1.1 - reordering +sorting

```
import pandas as pd
frame = pd.DataFrame(np.arange(12).reshape((4, 3)), index=[['a', 'a',
'b', 'b'], [1, 2, 1, 2]],
                     columns=[['Ohio', 'Ohio', 'Colorado'], ['Green',
'Red', 'Green']])
frame.index.names = ['key1', 'key2']
frame.columns.names = ['state', 'color']
# frame.swaplevel('key1', 'key2')
# frame.sort_index(level=0 )
# frame.swaplevel(0,1).sort_index(level=0 )
```

state		Ohio	Colorado	
color		Green	Red	Green
key1	key2			
a	1	0	1	2
	2	3	4	5
b	1	6	7	8
	2	9	10	11

CHAPTER 6.1.3 - indexing dataframe(set)

```
import pandas as pd
frame = pd.DataFrame({'a': range(7), 'b': range(7, 0, -1), 'c':
['one', 'one', 'one', 'two', 'two', 'two', 'two'],
                      'd': [0, 1, 2, 0, 1, 2, 3]})
# frame.set_index(['c'])
# frame.set_index(['c'], drop=False)
# frame.reset_index()
```

CHAPTER 6.2.1- data join

```
df1 = pd.DataFrame({'key': ['b', 'b', 'a', 'c', 'a', 'a', 'd'],
                    'data1': range(7)})
df2 = pd.DataFrame({'key': ['a', 'b', 'd'], 'data2': range(3)})
#merge 的时候有a and b so 有这个两个的value

pd.merge(df1, df2, on='key', how='inner')# on key 会叠加
```

	key	data1	data2
0	b	0	1
1	b	1	1
2	a	2	0
3	a	4	0
4	a	5	0
5	d	6	2

CHAPTER 6.2.2- data merge

```
left1 = pd.DataFrame({'key': ['a', 'b', 'a', 'a', 'b', 'c'], 'value':
range(6)})
right1 = pd.DataFrame({'group_val': [3.5, 7, 8]}, index=['a', 'b', 'c'])
pd.merge(left1, right1, left_on='key', right_index=True, how = 'outer')
```

	key	value	group_val
0	a	0	3.5
2	a	2	3.5
3	a	3	3.5
1	b	1	7.0
4	b	4	7.0
5	c	5	8.0

CHAPTER 6.2.3- data concatenate

```
import pandas as pd
s1 = pd.Series([0, 1], index=['a', 'b'])
s2 = pd.Series([2, 3, 4], index=['c', 'd', 'e'])
s3 = pd.Series([5, 6], index=['f', 'g'])
print(s1)
print()
print(s2)
```

```
print()
print(s3),
print()
pd.concat([s1, s2, s3],axis =1,keys=['one', 'two', 'three'])
```

```
a    0
b    1
dtype: int64
```

```
c    2
d    3
e    4
dtype: int64
```

```
f    5
g    6
dtype: int64
```

	one	two	three
a	0.0	NaN	NaN
b	1.0	NaN	NaN
c	NaN	2.0	NaN
d	NaN	3.0	NaN
e	NaN	4.0	NaN
f	NaN	NaN	5.0
g	NaN	NaN	6.0

Chapter 7.1 Groupby

```
import pandas as pd
df = pd.DataFrame({'key1' : ['a', 'a', 'b', 'b', 'a'], 'key2' :
['one', 'two', 'one', 'two', 'one'],
                  'data1' : [1,2,3,4,5], 'data2' : [1,2,3,4,5]})
#four key in the dataframe
df
# pieces = dict(list(df.groupby('key1')))) # 做dictionary
# df.groupby(['key1'])[['data2']].sum()
# df.groupby(len).sum() # string
```

	key1	key2	data1	data2
0	a	one	1	1
1	a	two	2	2
2	b	one	3	3
3	b	two	4	4
4	a	one	5	5

```
import pandas as pd

# Create DataFrame *****
data = {
```



```

    'fruit': ['apple', 'banana', 'apple', 'orange', 'banana',
'apple'],
    'quantity': [3, 2, 4, 1, 5, 2],
    'price': [0.5, 0.3, 0.5, 0.7, 0.3, 0.5]
}

```

```
df = pd.DataFrame(data)
```

```
# 1 Total Quantity Sold Per Fruit
```

```
total_quantity = df.groupby('fruit')[['quantity']].sum()
print("Total Quantity Sold Per Fruit:\n", total_quantity)
```

```
# 2 Total Revenue Per Fruit
```

```
df['total'] = df['quantity'] * df['price']
total_revenue = df.groupby('fruit')['total'].sum()
print("\nTotal Revenue Per Fruit:\n", total_revenue)
```

```
# 3 Average Price Per Fruit
```

```
average_price = df.groupby('fruit')['price'].mean()
print("\nAverage Price Per Fruit:\n", average_price)
```

```
Total Quantity Sold Per Fruit:
```

```

      quantity
fruit
apple         9
banana        7
orange        1

```

```
Total Revenue Per Fruit:
```

```

      fruit
apple    4.5
banana   2.1
orange   0.7
Name: total, dtype: float64

```

```
Average Price Per Fruit:
```

```

      fruit
apple    0.5
banana   0.3
orange   0.7
Name: price, dtype: float64

```

Chapter 7.2 : Data Aggregation

```
df.describe()
```

```

   fruit  quantity  price  total
0  apple         3    0.5    1.5
1  banana         2    0.3    0.6
2  apple         4    0.5    2.0

```

3	orange	1	0.7	0.7
4	banana	5	0.3	1.5
5	apple	2	0.5	1.0

```
tips = pd.read_csv("tips.csv")
tips.head()
tips.shape
```

```
tips['tip_pct'] = tips['tip'] / tips['total_bill'] # 加新的column
grouped = tips.groupby(["day", "smoker"])
list(grouped)
```

```
grp_pct = grouped['tip_pct']
grp_pct.agg('mean')
```

```
functions = ['count', 'mean', 'max']
result = grouped[['tip_pct', 'total_bill']].agg(functions)
```

```
grouped.agg({'tip' : np.max, 'size' : 'sum'})
```

day	smoker	
Fri	No	0.151650
	Yes	0.174783
Sat	No	0.158048
	Yes	0.147906
Sun	No	0.160113
	Yes	0.187250
Thur	No	0.160298
	Yes	0.163863

Name: tip_pct, dtype: float64

Chapter 7.3: Apply

```
def top(df, n=5, column='tip_pct'):
    return df.sort_values(by=column)[-n:] # 最后5个value
```

```
top(tips)
tips.groupby('smoker').apply(top)
```

```
result = tips.groupby('smoker')['tip_pct'].describe()
result
```

```
C:\Users\User\AppData\Local\Temp\ipykernel_13700\2792615095.py:5:
DeprecationWarning: DataFrameGroupBy.apply operated on the grouping
columns. This behavior is deprecated, and in a future version of
pandas the grouping columns will be excluded from the operation.
Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this
warning.
```

```
tips.groupby('smoker').apply(top)
```

	count	mean	std	min	25%	50%
75% \ smoker						
No	151.0	0.159328	0.039910	0.056797	0.136906	0.155625
0.185014						
Yes	93.0	0.163196	0.085119	0.035638	0.106771	0.153846
0.195059						
	max					
smoker						
No	0.291990					
Yes	0.710345					

Chapter 7.3.1-7.3.3: suppressing key,bucket analysis,fill with spcific group

Chapter 7.4.1: pivot table

```
tips.pivot_table(['size','tip', 'tip_pct', 'total_bill'],index=['day', 'smoker'])
tips.pivot_table(['tip_pct', 'size'], index=['time', 'day'],
columns='smoker', margins=True)
```

			size		tip_pct		
smoker			No	Yes	All	No	Yes
All							
time	day						
Dinner	Fri	2.000000	2.222222	2.166667	0.139622	0.165347	
0.158916							
	Sat	2.555556	2.476190	2.517241	0.158048	0.147906	
0.153152							
	Sun	2.929825	2.578947	2.842105	0.160113	0.187250	
0.166897							
	Thur	2.000000	NaN	2.000000	0.159744	NaN	
0.159744							
Lunch	Fri	3.000000	1.833333	2.000000	0.187735	0.188937	
0.188765							
	Thur	2.500000	2.352941	2.459016	0.160311	0.163863	
0.161301							
All		2.668874	2.408602	2.569672	0.159328	0.163196	
0.160803							

Chapter 7.4.2: cross tab

```
pd.crosstab([tips.time, tips.day], tips.smoker, margins=True)
```

smoker	No	Yes	All
time day			

Dinner	Fri	3	9	12
	Sat	45	42	87
	Sun	57	19	76
	Thur	1	0	1
Lunch	Fri	1	6	7
	Thur	44	17	61
All		151	93	244

CHAPTER 8.1 - Plotting and Visualization

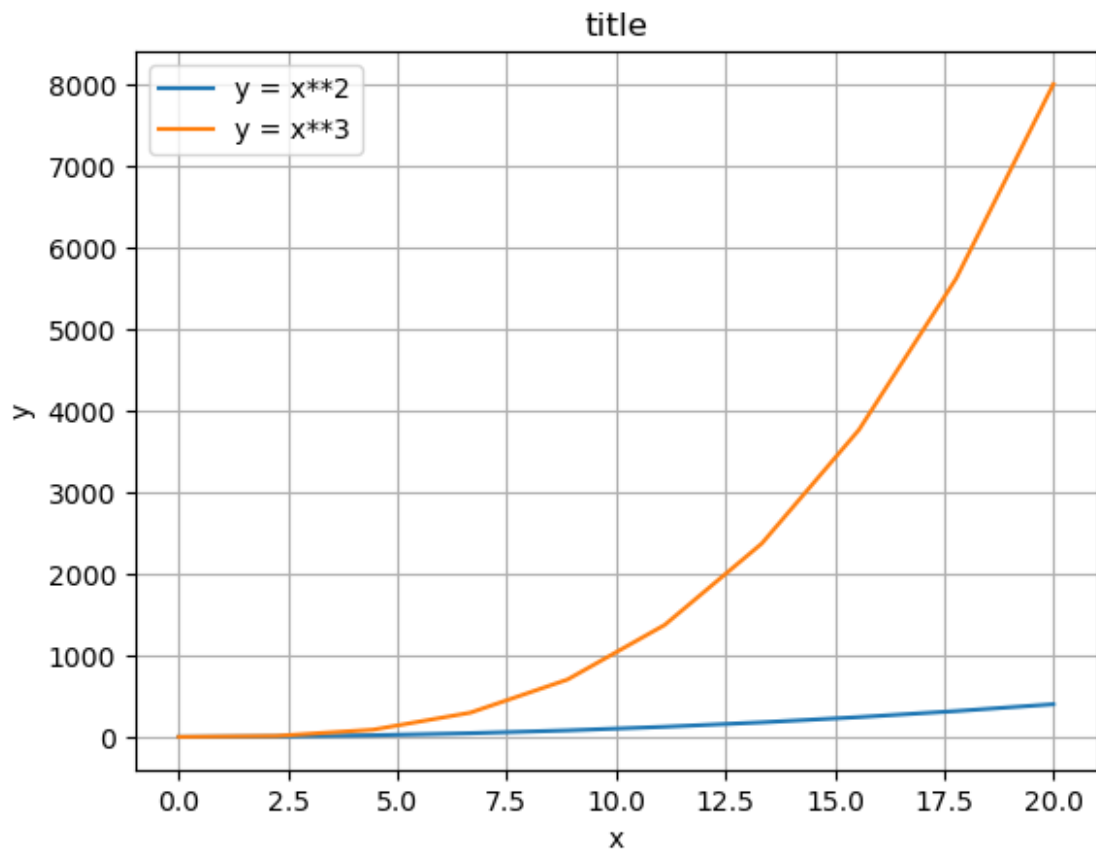
```
import matplotlib.pyplot as plt
%matplotlib inline

import numpy as np
import pandas as pd

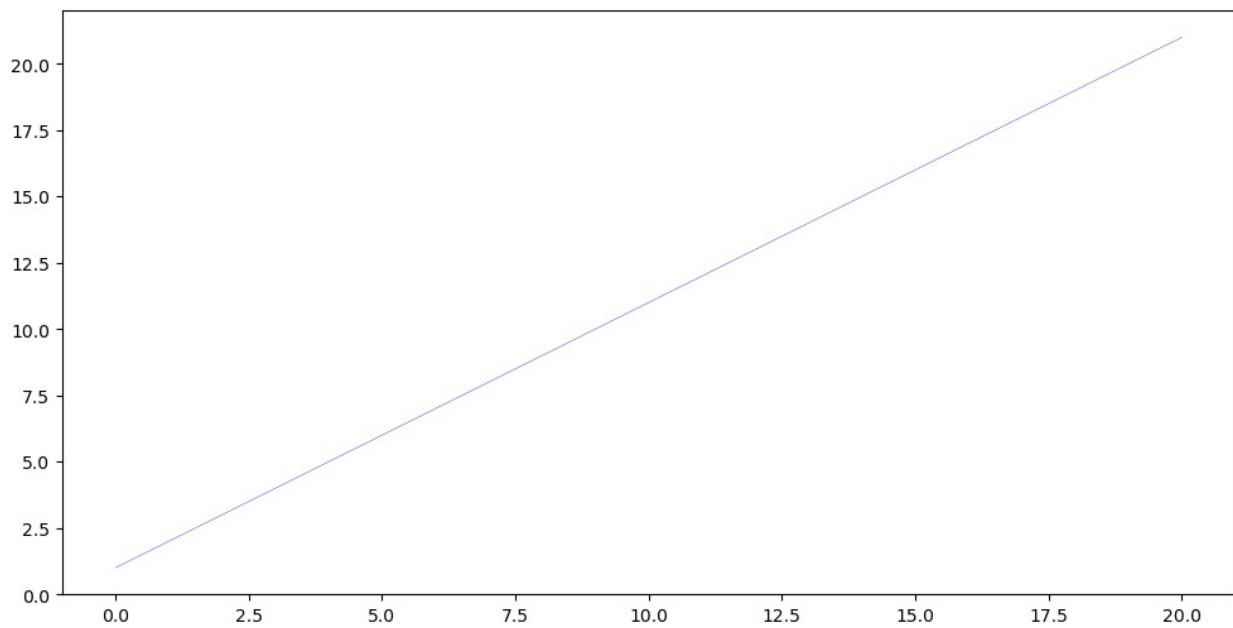
fig, ax = plt.subplots()

ax.plot(x, x**2, label="y = x**2")
ax.plot(x, x**3, label="y = x**3")
ax.legend(loc=2); # upper left corner # 内容不会遮挡到
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title('title');
ax.grid(True) # 放格子容易看

plt.show()
```



```
fig, ax = plt.subplots(figsize=(12,6))
ax.plot(x, x+1, color="blue", linewidth=0.25)
plt.show()
```

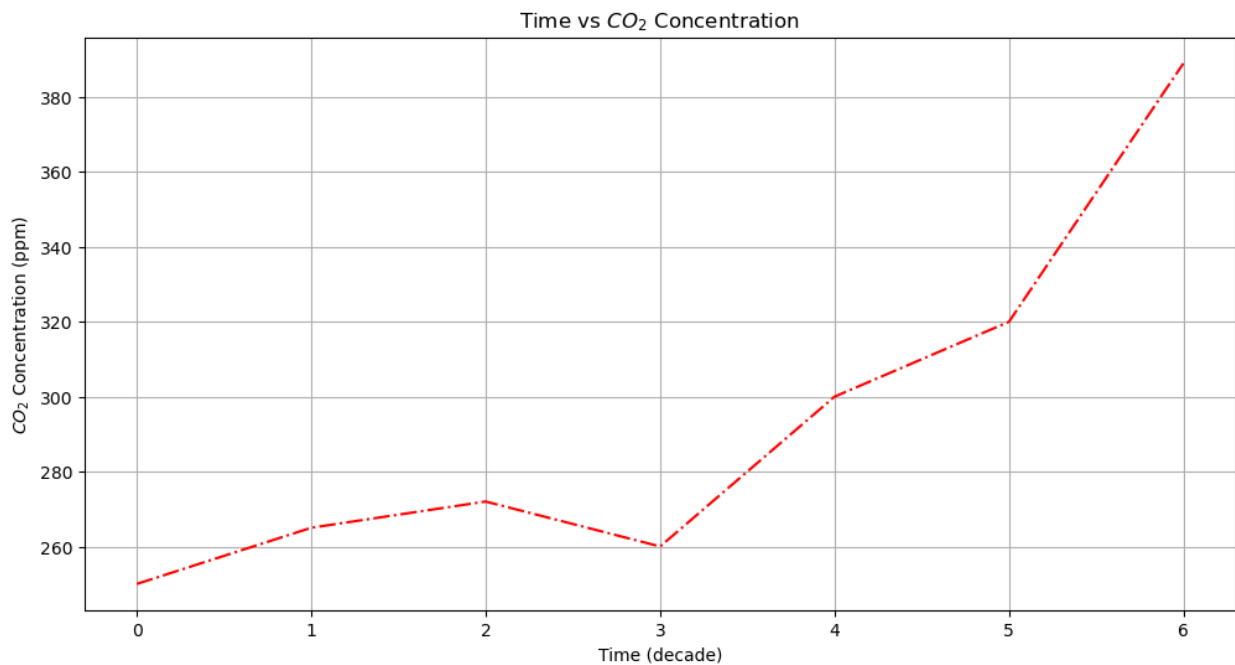


```

fig, EX1 = plt.subplots(figsize=(12,6))
x = [0,1,2,3,4,5,6]
y = [250, 265, 272, 260, 300, 320, 389]

EX1.plot(x,y, color='red', linestyle='-.')
EX1.set_title('Time vs $CO_2$ Concentration')
EX1.set_xlabel(r'Time (decade)')
EX1.set_ylabel(r'$CO_2$ Concentration (ppm)');
EX1.grid(True)
plt.show()

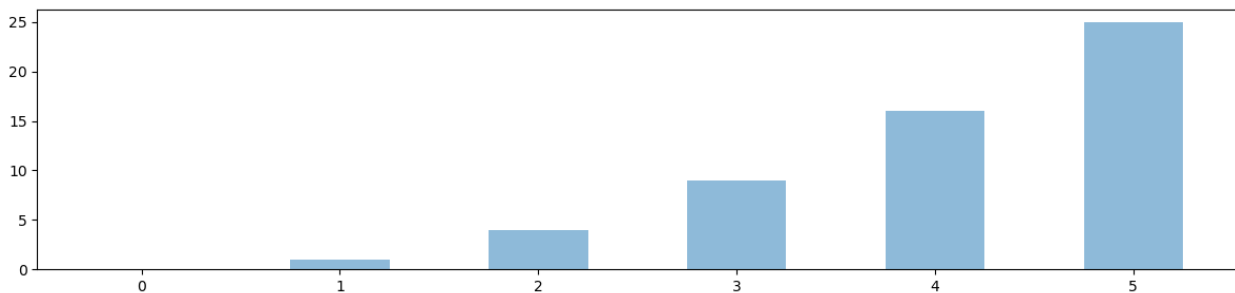
```



```

n = np.array([0,1,2,3,4,5])
fig, axes = plt.subplots(figsize=(12,3))
axes.bar(n, n**2, align="center", width=0.5, alpha=0.5)
fig.tight_layout()
plt.show()

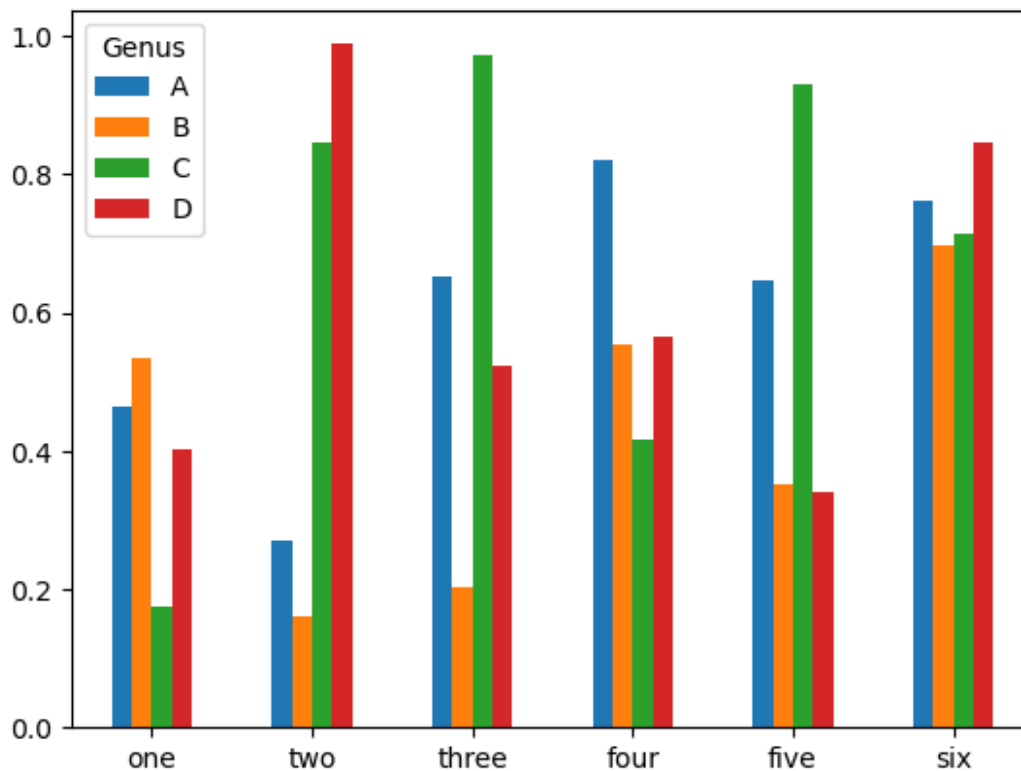
```



```
df = pd.DataFrame(np.random.rand(6, 4), index=['one', 'two', 'three',
'four', 'five', 'six'],
                  columns=pd.Index(['A', 'B', 'C', 'D'],
name='Genus'))
df
```

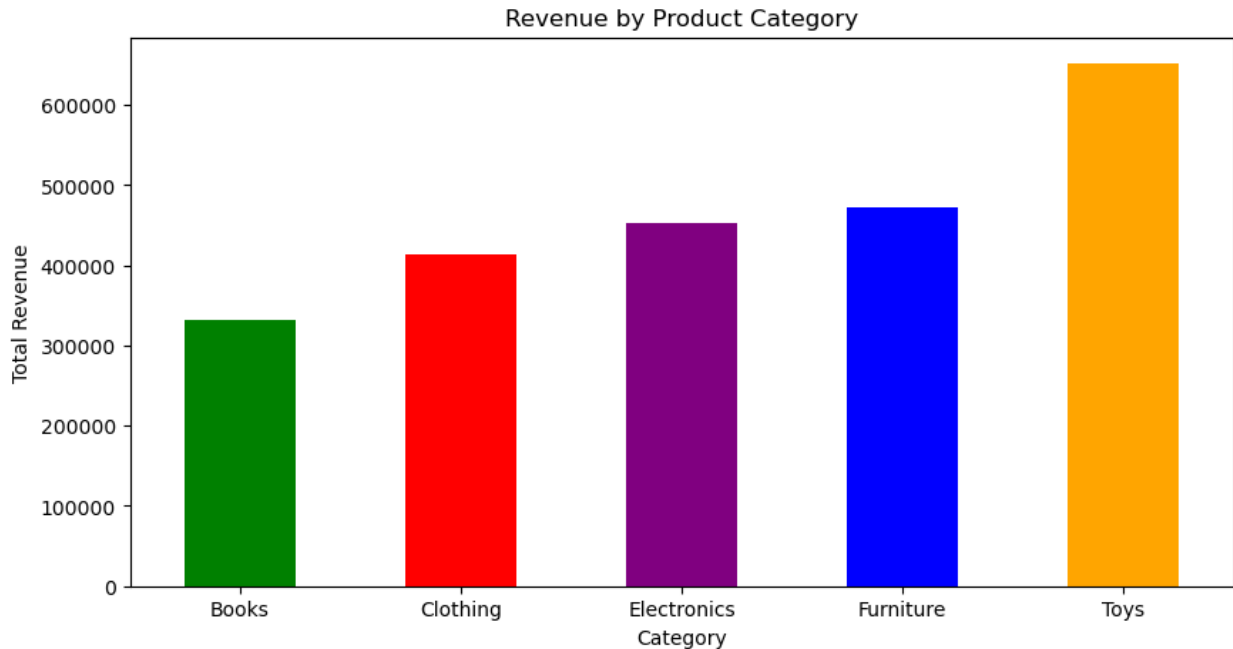
Genus	A	B	C	D
one	0.464151	0.534357	0.173575	0.402619
two	0.269096	0.161608	0.846373	0.988284
three	0.650901	0.201571	0.971407	0.523873
four	0.821417	0.553365	0.416678	0.565251
five	0.645200	0.351648	0.928947	0.340112
six	0.762367	0.695795	0.713780	0.845689

```
df.plot.bar()
plt.xticks(rotation =0)
plt.show()
```



```
import matplotlib.pyplot as plt
sales_data = pd.read_csv("products_sales.csv")
category_revenue = sales_data.groupby("Category")["Revenue"].sum()
plt.figure(figsize=(10,5))
category_revenue.plot.bar(color=['green', 'red', "purple", "blue", "orange"])
plt.title("Revenue by Product Category")
```

```
plt.xlabel("Category")
plt.ylabel("Total Revenue")
plt.xticks(rotation=0)
plt.show()
```



```
top_region = sales_data.groupby("Region")["Revenue"].sum().idxmax()
top_region_data = sales_data[sales_data["Region"] == top_region]
monthly_revenue = top_region_data.groupby("Month")["Revenue"].sum()

plt.figure(figsize=(10,5))
plt.plot(monthly_revenue.index, monthly_revenue.values, marker="o",
linestyle='dashed', color='purple')
plt.title(f"Monthly Revenue Trend for {top_region}")
plt.xlabel("Month")
plt.ylabel("Total Revenue")
plt.grid(True)
plt.tight_layout()
plt.show()
```